

Claude Unleashed

Autonomous-agent orchestration platform

Launch and supervise Claude Code sessions as autonomous background jobs — from a Mac app, CLI, or Slack.

ENGINEERING DEEP-DIVE

v1.5.1 GA

daemon · CLI · Mac app · MCP

Michael Starukhin · Software Architect (OMS / B2B)





What Claude Unleashed is

A long-running **daemon** supervises Claude Code workers running as background jobs — in isolated git worktrees or on your live branch — driven by a thin CLI, a SwiftUI Mac app, and MCP tools.

Run agents unattended

Sessions execute, self-extend turns, and reach terminal state — no babysitting.

Compose, don't script

Agents → profiles → workflows → swarms → recipes. Reusable, shareable YAML.

Supervised by design

Overseer watches every session; approvals gate risky tools; Slack keeps you in the loop.

PRINCIPLE

We use claude-unleashed to build claude-unleashed — every shipped capability is retargeted onto the development loop itself.

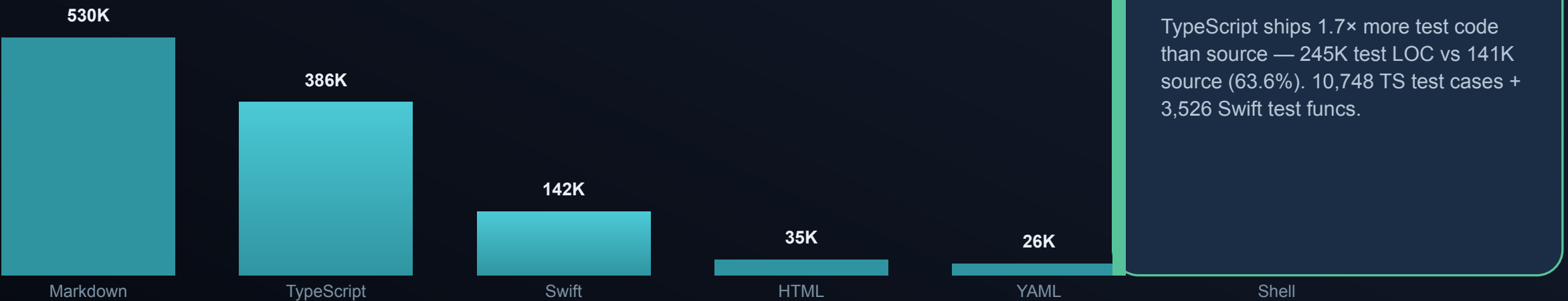


Engineering at a glance

~8.5 weeks · 26 contributors · built with *claude-unleashed* itself.



Lines of code by language (K)



TESTS WITH EVERYTHING

TypeScript ships 1.7× more test code than source — 245K test LOC vs 141K source (63.6%). 10,748 TS test cases + 3,526 Swift test funcs.



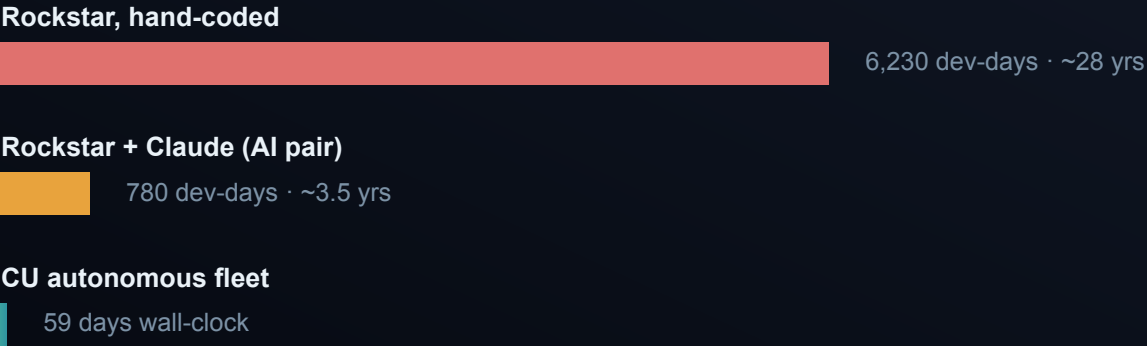
Development velocity

623K lines of tested, reviewed code in 59 days — work that traditionally scopes in dev-years.

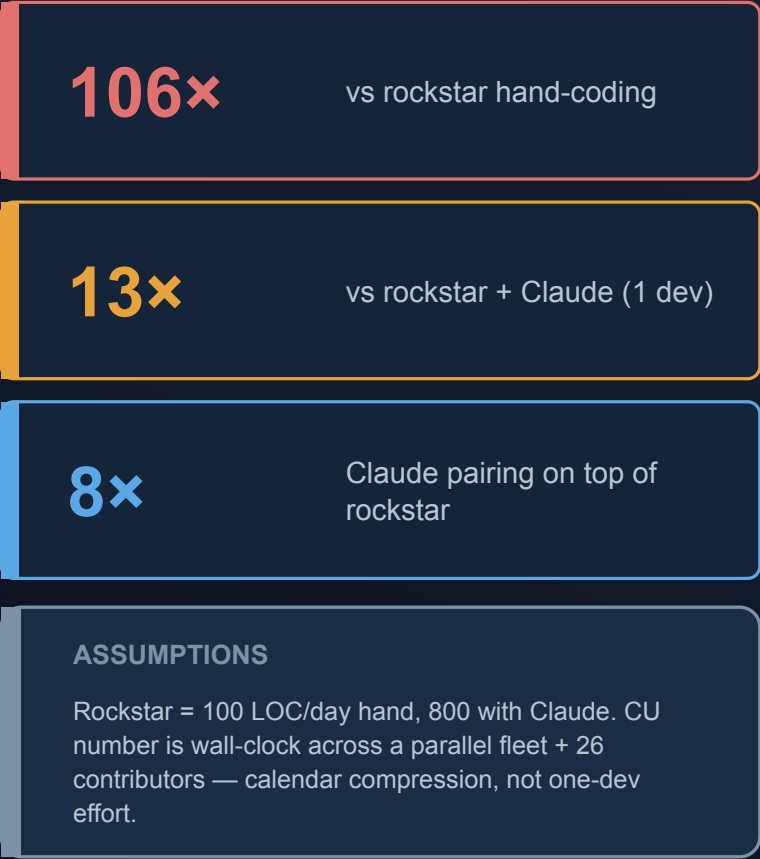
MEASURED



Effort to build the SAME 623K LOC (dev-days)



MODELED — SPEEDUP vs CU 59-DAY CALENDAR





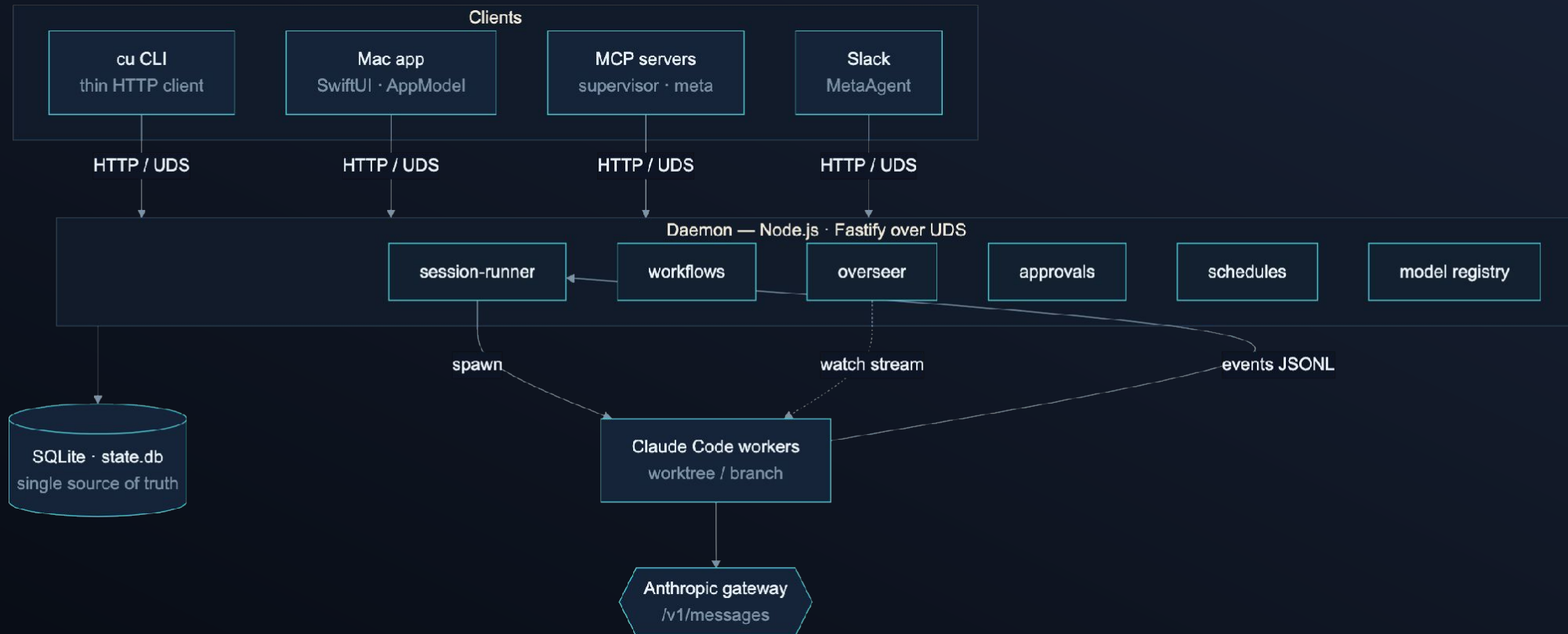
01

Architecture

How the daemon, clients, workers, and gateway fit together.



System topology



BOUNDARY

The daemon owns all SQLite state and the full agent lifecycle. CLI, Mac app, and MCP are stateless transports over the UDS socket.



Module boundaries

- ▶ **daemon** — owns persistent state (SQLite) + agent lifecycle. No other package touches the DB.
- ▶ **CLI** — thin HTTP client over UDS. `runCli()` returns a result object — never `process.exit`.
- ▶ **Mac app** — thin UDS client over an `@Observable AppModel`. No business logic.
- ▶ **core** — dependency-free shared types + event schema. Imports from nothing.
- ▶ **mcp-supervisor** — translates MCP tool calls → daemon HTTP. Stateless.

Clients — CLI · Mac app · MCP

HTTP / UDS transport

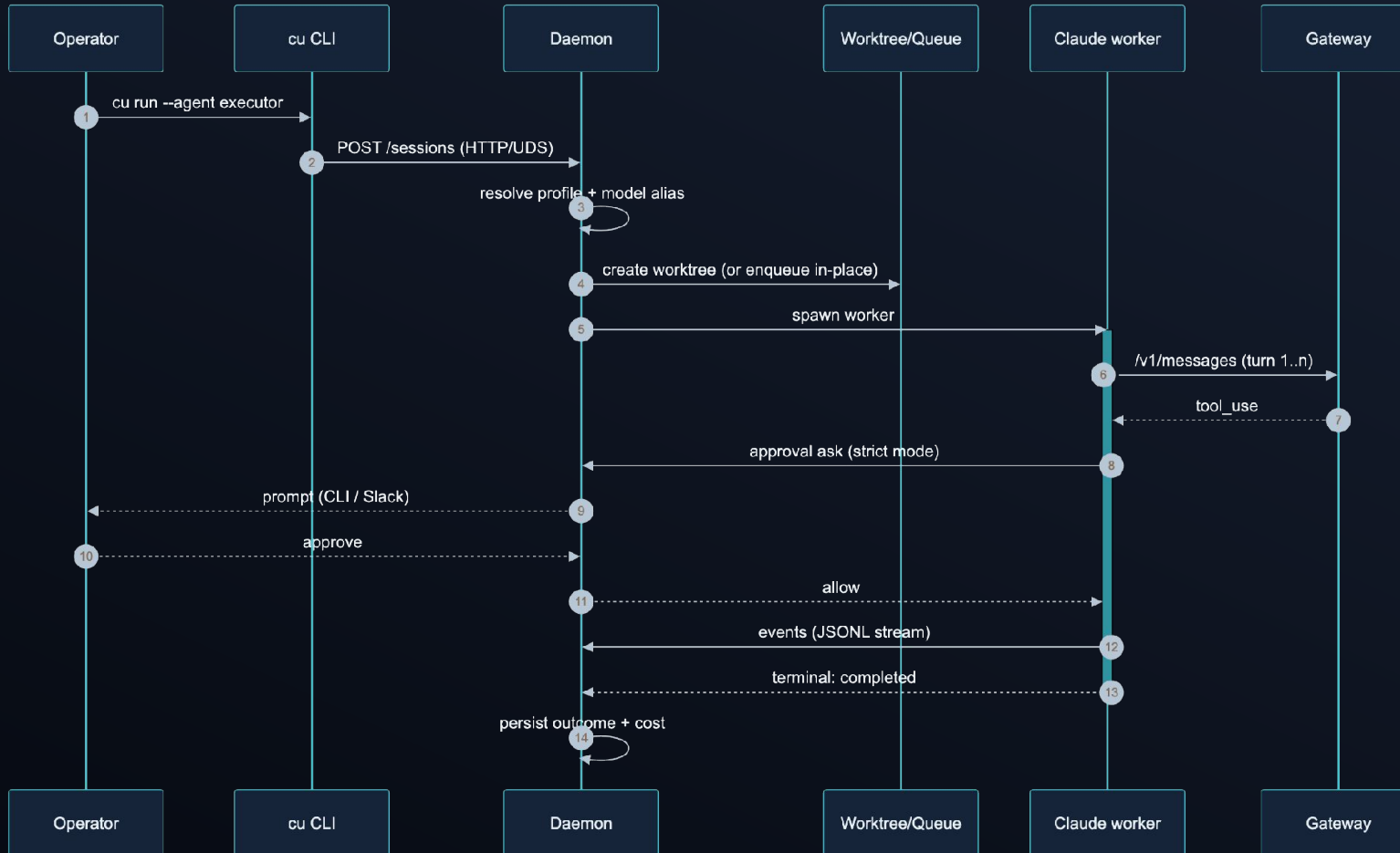
Daemon — lifecycle + orchestration

core — shared types (no deps)

SQLite — persistent state



Anatomy of a launch



- **UDS, not TCP** — every client call is a Unix-domain-socket HTTP request.
- **Resolve first** — profile + model alias resolve before any worker spawns.
- **Isolate** — worktree created — or the in-place queue admits the run.
- **Stream** — worker emits a JSONL event stream the daemon persists.
- **Gate** — strict mode pauses on tool-use until a human approves.



Token & model resolution

Auth resolves through a strict priority cascade; the model registry self-heals against gateway drift.

1

process.env ANTHROPIC_AUTH_TOKEN

operator-set (shell / plist)

2

~/ .claude/settings.json

env.ANTHROPIC_AUTH_TOKEN

3

apiKeyHelper

DevBar protocol shell command

4

AI Suite manager

auto-detection

verify the registry

```
$ curl --unix-socket daemon.sock \  
    http://localhost/models | jq .source
```

```
"live"    probe succeeded
```

```
"fallback" restart w/ gateway env
```

WHY ALIASES

Agent + profile YAMLs reference opus / sonnet / haiku — never hardcoded IDs. A rotated gateway model id can't break a dispatch; the registry re-probes and heals.



02

Sessions

Two execution modes · lifecycle · queue serialization.



Two execution modes

Worktree mode *default*

- ▶ **Full isolation** — each session in `~/.claude-unleashed/worktrees/<id>/.`
- ▶ **True parallelism** — N sessions, zero cross-talk; auto-pruned when terminal.

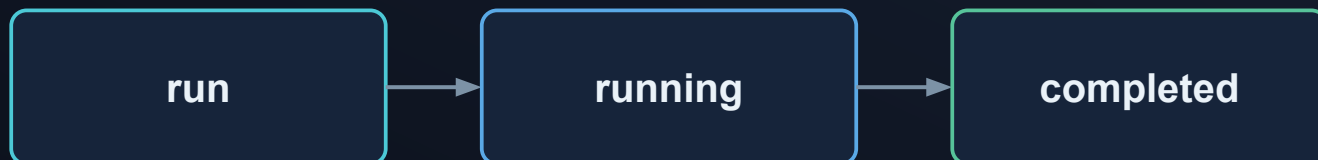
Branch mode *runMode: in-place*

- ▶ **On your live branch** — serialized per-repo via InPlaceQueue — no disposable worktrees.
- ▶ **Clean-tree contract** — clean in, clean out; cancel-safe at three checkpoints.





Lifecycle & state model



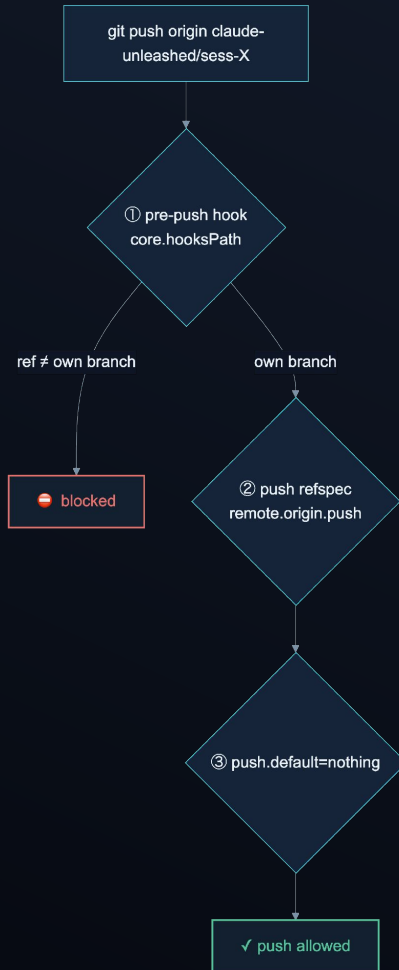
Terminal & recovery states



- ▶ **Verbs** — run · continue · tail · kill · pause · resume · extend-turns — all on the CLI/HTTP surface.
- ▶ **Persisted per session** — turn count, cost (USD), worker PID, branch, dedup hash, prior-turn outcome.
- ▶ **Boot sweep** — leaked running/queued rows whose worker PIDs are gone are crashed at daemon boot (migration 0034).



Cross-session push protection



- ▶ **① Pre-push hook** — authoritative; via `core.hooksPath`. Blocks any `claude-unleashed/*` ref that isn't this session's branch.
- ▶ **② Push refspec** — `remote.origin.push` limits bare git push to the own branch.
- ▶ **③ `push.default=nothing`** — kills fallback `matching/simple/current` semantics.

AUTO-PRUNE

Terminal sessions' worktrees reaped every 6h with a 24h grace; dirty trees keep a preserved-work marker.



03

Composition

Agents · groups · profiles · workflows · swarms · recipes.



Agents



The canonical **full-process** pipeline — plan → execute → review

agent.yaml

```
name: executor
model: sonnet # alias, self-heals
archetype: developer
allowedTools:
  [Read, Edit, Bash, Grep, Task]
disallowedTools: [WebFetch]
systemPrompt: |
  Implement the plan via strict TDD...
```

- ▶ **YAML contract** — model · systemPrompt · tool allowlist · archetype · caps.
- ▶ **Aliases only** — opus/sonnet/haiku so dispatches survive gateway rotation.
- ▶ **Catalog** — built-in planner/executor/reviewer + il-* / inv-* role archetypes.
- ▶ **recommendedSkills** — filtered to what's installed in the dispatch context.



Agent-groups & profiles

Agent-groups

Bundle a catalog of subagents under one reusable name, with an optional coordinator.

`review-team`

`coordinator``code-reviewer``security-reviewer``domain-reviewer``premortem`

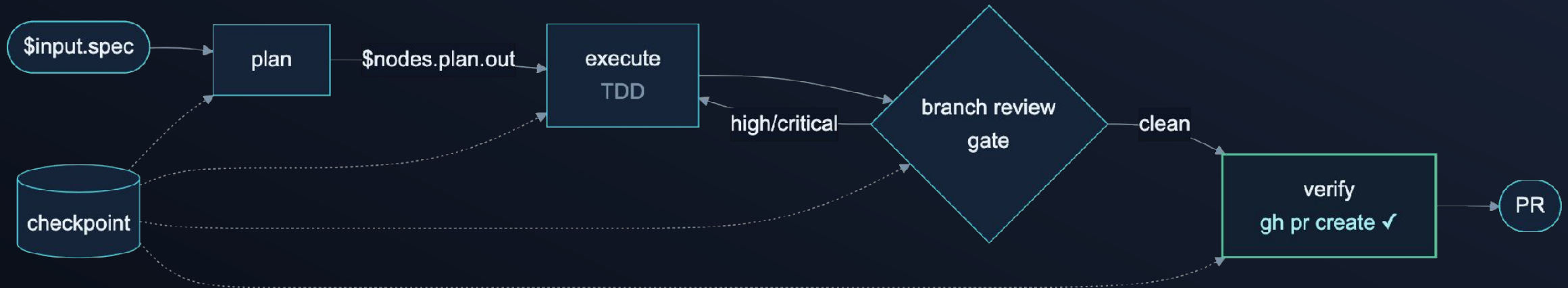
Profiles

Reusable launch configs — bind agent + model + caps + permission mode to a name or repo.

`agent / agentGroup``model``allowedTools / disallowedTools``maxTurns``maxBudgetUsd``permissionMode``swarm / swarmParams`



Workflows — multi-node DAGs



► **Typed node I/O** — `$input.*` and `$nodes.<id>.field` thread outputs forward.

► **Deterministic branching** — when: conditions; dry-run compiles topology with no worker.

► **Single-PR invariant** — only verify opens a PR — closes the duplicate-PR failure class.

► **Resume recovery** — resume from checkpoint; re-run only the failed node, siblings preserved.



Swarms — parameterized fan-out

Reusable, deterministically-branching agent fan-out blocks. A workflow node hosts a swarm; only its `return:` is visible to the host.

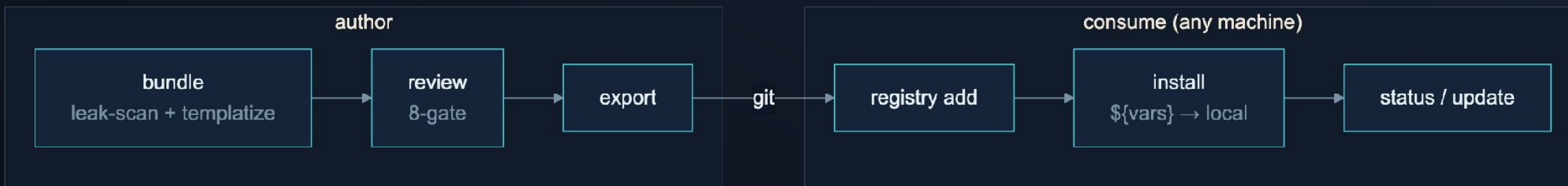


forEach: parallelize over a discovered list **when:** branch on conditions surfaced in **cu actions**



Recipes — share a CU setup

Git-shareable bundles of profiles · agents · groups · workflows · swarms · schedules · CDC subs. Machine-specific literals are abstracted to `${variables}` at bundle time and re-bound on install.



recipe.yaml manifest

- ▶ **name · version** — semver, no path separators
- ▶ **variables** — type + detect strategy + default
- ▶ **artifacts** — inventory across 7 stores
- ▶ **requires** — skills + MCP servers declared

`${variable}` binding

`BUNDLE abstractText() → ${var}`

`INSTALL detect → bindText()`

- ▶ **5 types** — path · gus-org · sf-id · cron · string
- ▶ **auto-detect** — git-toplevel · sf-whoami · cu-doctor
- ▶ **lockfile** — resolved ref + bound values + hashes

8 review gates — block on any error

- | | |
|-------------------|-------------------|
| ✓ leak-scan | ✓ cdc-leak |
| ✓ cross-ref | ✓ requirements |
| ✓ artifact-schema | ✓ version-tag |
| ✓ name-length | ✓ undeclared-file |



Skills we ship for driving CU

18 skills install into the Claude Code harness — so Claude (and you) can author, launch, and operate CU by name.

Authoring

7 skills

- `cu-agent-authoring`
- `cu-agent-groups`
- `cu-profile-authoring`
- `cu-workflow-authoring`
- `cu-swarm-authoring`
- `cu-schedule-authoring`
- `cu-gus-cdc`

Running & operating

5 skills

- `cu-session-launch`
- `cu-session-management`
- `cu-session-monitor`
- `cu-multi-session-orchestra`
`tion`
- `cu-workspaces`

Orientation & app

3 skills

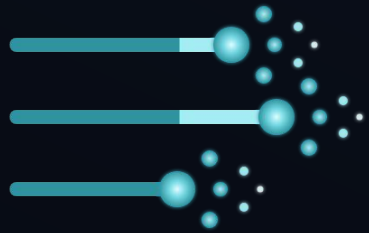
- `cu-getting-started`
- `cu-cli`
- `cu-app-setup`

Ship & maintain

3 skills

- `recipe-export-review`
- `filing-cu-issues`
- `claude-unleashed`

Installed via the `cu-cli` plugin; agents pull them in through `recommendedSkills` (unreachable entries are filtered before the run).



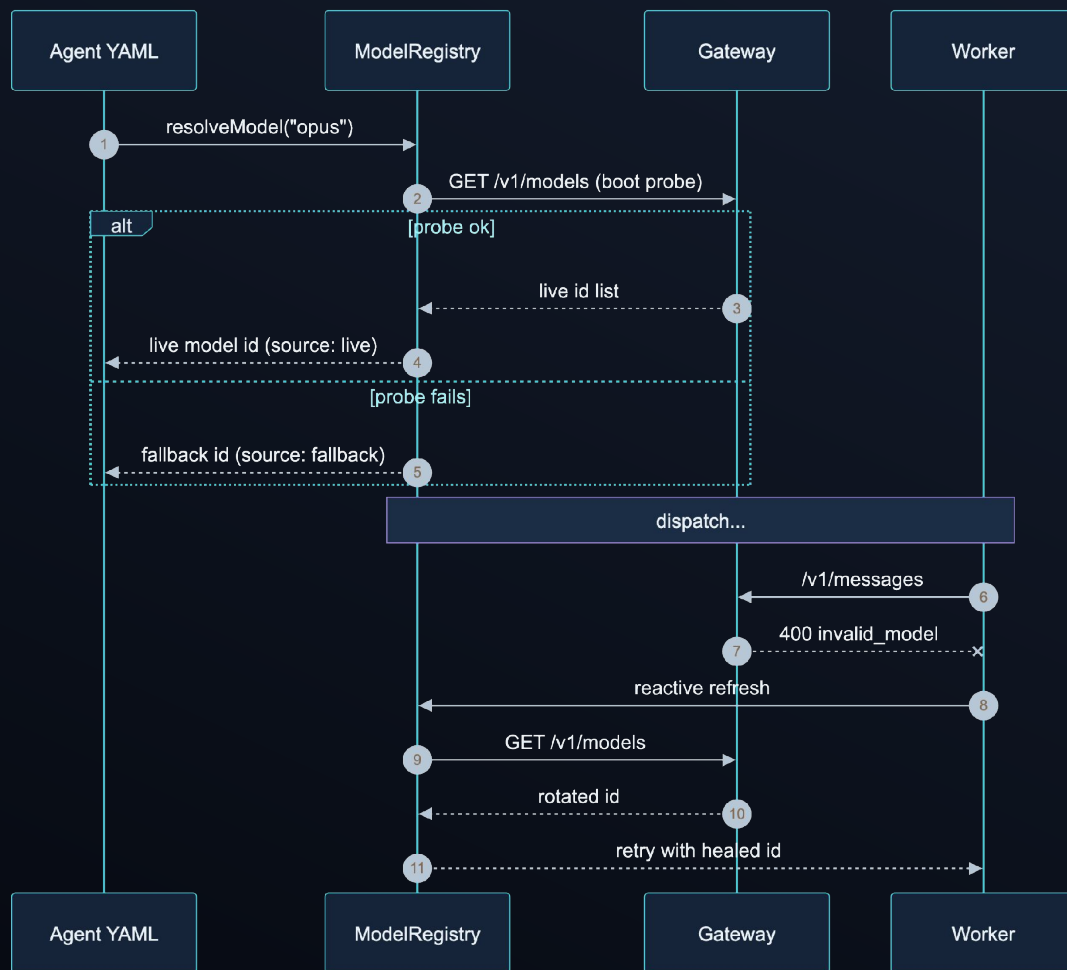
04

Reliability & Control

Model self-heal · the Overseer · approvals.



Model registry — self-healing



- **Boot probe** — GET /v1/models maps aliases → live ids; falls back to a static ladder.
- **Reactive refresh** — a 400 invalid_model triggers a re-probe + retry — no manual restart.
- **source flag** — /models reports live · cache · fallback.

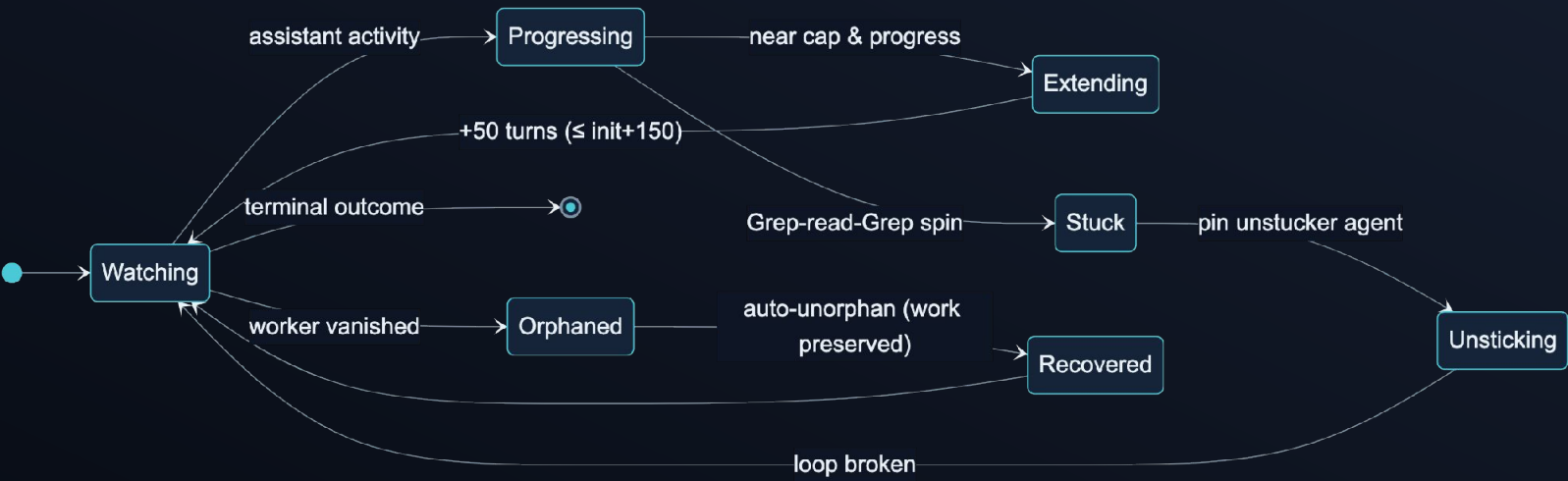
INCIDENT-BORN

A gateway phantom-id wave (500s across dispatches) drove this design — YAMLS now self-heal like profile resolution.



The Overseer

Watches every session's event stream in real time and intervenes — so a run finishes instead of stalling or dying at the cap.



Turn extension

+50 mid-run on progress ($\leq \text{init}+150$) — ADR 0022.

Auto-unstick

detects spin loops, pins an unstucker agent.

Stuck guard

false-positive guard spares working sessions.

Auto-unorphan

recovers vanished workers; preserves work.

Every extension decision is audit-logged per session in *overseer.log*.



Approvals & permissions

default

prompt on non-allowlisted

acceptEdits

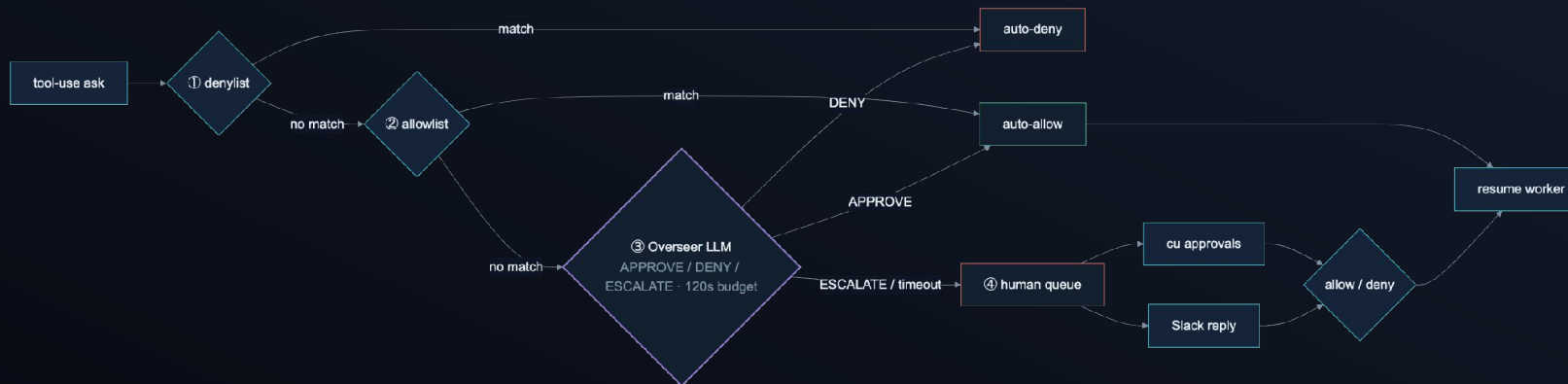
auto-accept file edits

plan

read-only planning

bypassPermissions

skip all prompts



OVERRIDES

Session-scoped allowedTools / disallowedTools JSON lists override catalog defaults; operators author policies under `~/.claude-unleashed/policy/`.

CASCADE

denylist → allowlist → Overseer LLM (APPROVE/DENY/ESCALATE, 120s budget) → human. Operator questions always escalate — never auto-answered.

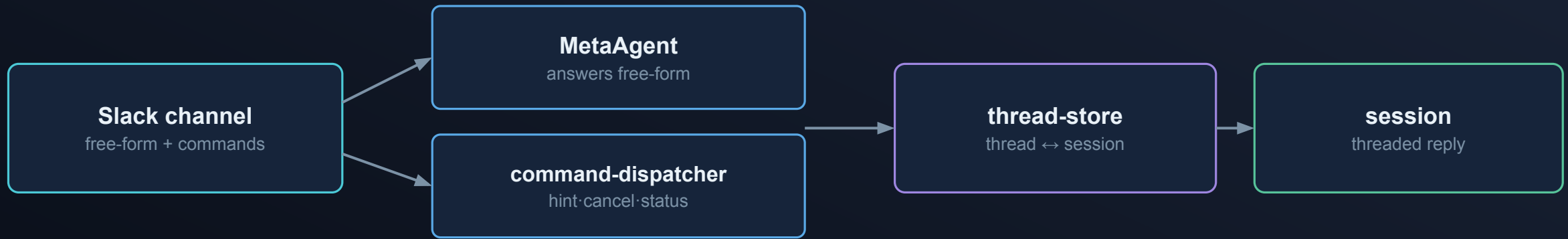


05

Integrations

Slack · Salesforce Workspaces · GUS CDC · schedules.

Slack integration



- ▶ **Session notifier** — posts start/stop, opens a thread; hints + approvals thread as replies.
- ▶ **Rate-limit resilient** — exponential backoff + token refresher; Enterprise Grid via per-workspace fan-out.
- ▶ **Contract** — channel-vs-thread routing, orphan-thread handling, confirm-gate for spawn (ADR 0028).



Salesforce Workspaces

Lease Salesforce cloud dev environments as **remote workers** — workers run inside the workspace container, not your laptop.

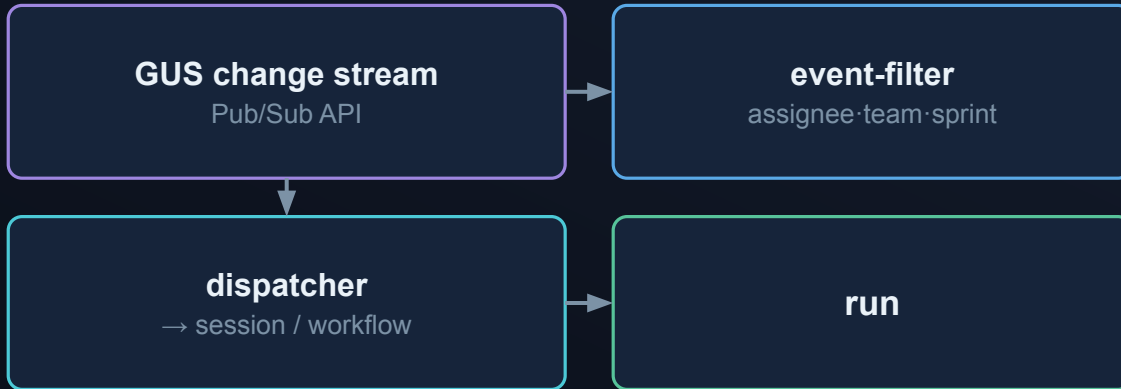


- ▶ **RemoteWorkerLauncher** — spawns the worker in-workspace; sessions track `execution_target = local | sfw`.
- ▶ **Lifecycle managed** — provision, warm-start (reaped off the warm clock), sync, auto-reap.
- ▶ **Version-gated** — daemon refuses the feature unless `sfworkctl ≥ 1.8.1`.



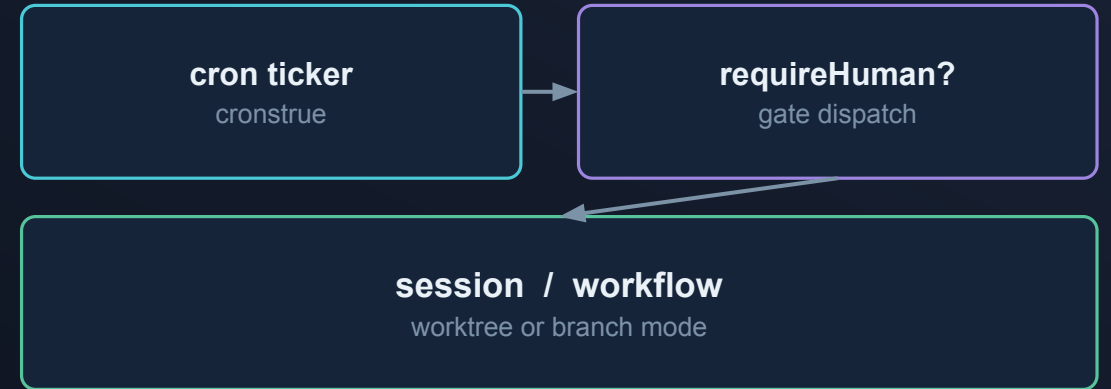
Event-driven triggers

GUS CDC subscriptions



- ▶ **Trigger on real work** — bug/story changes fire a session — e.g. auto-investigate on triage.
- ▶ **Resilient stream** — backoff on auth failure; degraded / recovered events surfaced.

Schedules & wake-on-schedule



- ▶ **Cron-driven** — binds an agent / group / profile to a cron expression.
- ▶ **requireHuman** — blocks auto-dispatch until you approve via CLI/Slack.



06

Foundation & Quality

Storage · CLI driveability · the release gate.



Storage — one SQLite DB

~/.claude-unleashed/state.db (prod) · -dev variant. Additive-only migrations run at boot; the daemon owns every write.

sessions	events	approvals	workflows
schedules	profiles	agents	agent-groups
swarms	recipes	policies	slack-threads
gus-cdc subs	workspaces	migrations	anomalies

DISCIPLINE

No other package reads the DB directly. State diverges instantly if dev and prod cross streams mid-iteration — they share nothing.



CLI & observability

```
$ cu dashboard --live
```

```
RUNNING 3    QUEUED 1    DONE 47
```

```
● a1f3 executor feat/auth t142/250
```

```
● b8c2 reviewer fix/race t083/200
```

```
⦿ c4d9 planner spec/cdc queued
```

```
$ cu sessions ls --json | jq .
```

```
[{ "id":"a1f3","status":"running" }]
```

- ▶ **cu dashboard --live** — interactive Ink TUI — running sessions, worktrees, live event stream.
- ▶ **--json everywhere** — machine-readable for chaining + agent-driven smoke tests.
- ▶ **70+ commands** — run · workflow · sessions · approvals · recipes · swarm · schedules · doctor ...
- ▶ **Principle** — every surface testable + driveable via the public path — not internal helpers.



Three-tier release gate



Tier 1 · Artifacts

DMG sig & size · skill URLs · tarball

Tier 2 · Runtime

real Fastify+SQLite+worker — the gate;
every behavior PR adds one

Tier 3 · Install loop

fresh install + binary invocation

`pnpm e2e` runs all three (~5 min); `scripts/release.sh` runs the same gate and refuses to advance if any tier fails.

The whole picture



Compose

- agents
- agent-groups
- profiles
- workflows
- swarms
- recipes

Run & supervise

- worktree / branch
- Overseer
- approvals
- model self-heal

Integrate

- Slack
- SF Workspaces
- GUS CDC
- schedules (cron)

Trust

- 3-tier gate
- push protection
- SQLite truth
- driveable CLI



Thank you

Claude Unleashed · autonomous agents, supervised by design

REPOSITORY

`git.soma.salesforce.com/cc-oms/claude-unleashed`

SLACK

`#claude-unleashed`

Michael Starukhin · Software Architect (OMS / B2B)